

Modelagem de Redes Complexas no Metagame de Yu-Gi-Oh!: Análise de Coocorrência, Detecção de Comunidades e Centralidade

Matheus Ferreira Unten

¹Instituto de Computação (IComp) – Universidade Federal do Amazonas (UFAM)
Av. Gen. Rodrigo Octávio, 6200, Coroado I – 69080-900 – Manaus – AM – Brasil

matheus.untен@icomp.ufam.edu.br

Abstract. *This paper presents a graph-theoretic modeling of the competitive metagame of the trading card game Yu-Gi-Oh!, framed as a weighted co-occurrence network. Cards are represented as vertices, and an edge between two cards indicates their simultaneous presence in simulated competitive decklists. Edge weights are computed both as raw co-occurrence frequencies and as Jaccard similarity indices to mitigate the distorting effect of universally popular cards. The resulting graph $G = (V, E, W)$ with $|V| = 212$ vertices and $|E| = 3,426$ edges is analyzed via weighted PageRank, betweenness centrality, and the Louvain community detection algorithm. Experiments identify staples – cards of high cross-archetypal importance – and automatically recover the archetype structure of the metagame, achieving a modularity of $Q = 0.3094$ across 15 communities, without any prior semantic labeling of card functions.*

Resumo. *Este artigo apresenta uma modelagem em teoria dos grafos do metagame competitivo do jogo de cartas colecionáveis Yu-Gi-Oh!, representado como uma rede de coocorrência ponderada. Cartas são vértices, e uma aresta entre duas cartas indica sua coexistência em decklists competitivas simuladas por um modelo probabilístico de lei de potência (Zipf). Pesos de arestas são calculados como frequência bruta de coocorrência e como índice de Jaccard normalizado. O grafo resultante $G = (V, E, W)$ com $|V| = 212$ vértices e $|E| = 3.426$ arestas é analisado por PageRank ponderado, centralidade de intermediação e pelo algoritmo de Louvain para detecção de comunidades. Os experimentos identificam automaticamente as staples (cartas de alta relevância inter-arquetípica) e recuperam a estrutura de arquétipos do metagame, alcançando modularidade $Q = 0,3094$ em 15 comunidades, sem qualquer rotulação semântica prévia das cartas.*

1. Introdução

Jogos de cartas colecionáveis (*Trading Card Games*, TCGs) como Yu-Gi-Oh!, Magic: The Gathering e Pokémon TCG possuem ecossistemas competitivos altamente complexos, denominados *metagame*. O metagame de Yu-Gi-Oh! é composto por centenas de *arquétipos* – grupos temáticos de cartas que interagem sinergicamente – e por *staples*: cartas de utilidade geral que transitam entre múltiplos arquétipos, tornando-se ubíquas no cenário competitivo.

A construção de um baralho competitivo (*deck*) não é aleatória. Ela é orientada por um conjunto de restrições implícitas: quais cartas têm sinergia entre si, quais recursos são compartilháveis entre arquétipos e quais peças são indispensáveis independentemente do estilo de jogo. Identificar estas estruturas manualmente requer conhecimento especializado do domínio e análise de milhares de listas de torneio.

Problema central: como extrair automaticamente, por métodos puramente matemáticos e sem qualquer conhecimento semântico prévio das regras do jogo, as cartas mais influentes do metagame e os agrupamentos de cartas que apresentam alta sinergia entre si?

A hipótese deste trabalho é que a estrutura do metagame pode ser capturada integralmente por um modelo de grafo de coocorrência, e que as ferramentas clássicas de análise de redes complexas – centralidade de PageRank, centralidade de intermediação (*betweenness*) e detecção de comunidades pelo algoritmo de Louvain [Blondel et al. 2008] – são suficientes para recuperar os padrões que um especialista identificaria manualmente.

Este trabalho contribui com: (i) a definição formal do problema de análise de metagame como um problema de grafos; (ii) um modelo probabilístico baseado na distribuição de Zipf para simular decklists realistas de torneio; (iii) a implementação completa do pipeline em Python com NetworkX [Hagberg et al. 2008]; e (iv) análise experimental dos resultados com tabelas e discussão das métricas obtidas.

O restante do artigo está organizado da seguinte maneira: a Seção 2 descreve a modelagem teórica em grafos; a Seção 3 apresenta os algoritmos e seu embasamento teórico; a Seção 4 descreve a implementação; a Seção 5 apresenta os experimentos e resultados; a Seção 7 conclui o trabalho.

2. Modelagem do Problema em Grafos

2.1. Definição Formal

O metagame de Yu-Gi-Oh! é modelado como um grafo não direcionado e ponderado $G = (V, E, W)$, onde:

- **Conjunto de vértices** V : cada vértice $v \in V$ representa uma carta única no universo do jogo (e.g., *Ash Blossom & Joyous Spring*, *Blue-Eyes White Dragon*). Cada vértice carrega um atributo de frequência $\text{freq}(v)$, definido como o número total de decklists nas quais a carta v aparece.
- **Conjunto de arestas** E : uma aresta $e = \{u, v\} \in E$ é criada se e somente se as cartas u e v coocorrem em pelo menos θ decklists simuladas (limiar $\theta = 2$ neste trabalho).
- **Pesos** W : cada aresta $\{u, v\}$ recebe dois pesos:
 1. **Peso bruto** $w(u, v)$: a contagem absoluta de decklists nas quais u e v coocorrem.
 2. **Índice de Jaccard** $J(u, v)$: uma métrica normalizada que penaliza pares envolvendo cartas excessivamente comuns:

$$J(u, v) = \frac{\text{freq}(u, v)}{\text{freq}(u) + \text{freq}(v) - \text{freq}(u, v)} \quad (1)$$

onde $\text{freq}(u, v)$ é a coocorrência bruta de u e v .

O índice de Jaccard (Equação 1) é essencial para evitar que cartas universalmente comuns (*staples*) criem arestas de peso desproporcional com todas as demais cartas, o que distorceria tanto a visualização quanto as métricas de centralidade.

2.2. Exemplo Ilustrativo

Considere um cenário reduzido com três cartas simuladas em um deck do arquétipo Branded: $V_{\text{deck}} = \{\textit{Fallen of Albaz}, \textit{Branded Fusion}, \textit{Ash Blossom}\}$.

As arestas de coocorrência geradas são: $\{\textit{Fallen of Albaz}, \textit{Ash Blossom}\}$, $\{\textit{Branded Fusion}, \textit{Ash Blossom}\}$, e $\{\textit{Fallen of Albaz}, \textit{Branded Fusion}\}$. O peso bruto de cada uma é incrementado em +1.

Como *Ash Blossom* também é amostrada em decks de outros arquétipos (Kashtira, Swordsoul, Labrynth, etc.), seu vértice acumula alta centralidade de intermediação, tornando-se um *hub* estrutural que conecta comunidades arquetípicas distintas. Esta emergência natural – sem injeção manual – é o fenômeno central que o modelo é projetado para capturar.

2.3. Propriedades Esperadas da Rede

Redes de coocorrência derivadas de domínios culturais ou competitivos frequentemente exibem propriedades de redes complexas [Barabási and Albert 1999]:

- **Distribuição de grau em lei de potência:** pouquíssimas cartas (*hubs*) possuem grau muito elevado, enquanto a maioria das cartas tem baixo grau – reflexo direto da hierarquia de uso no metagame.
- **Estrutura de comunidades:** cartas de um mesmo arquétipo formam cliques densos entre si, ao passo que as conexões entre arquétipos distintos são mediadas pelas *staples*.
- **Mundo pequeno:** o diâmetro médio da rede é pequeno em relação ao número de vértices, dada a alta conectividade gerada pelas *staples* transversais.

3. Algoritmos e Fundamentação Teórica

3.1. Simulação Probabilística com Distribuição de Zipf

Para construir as coocorrências, é necessário simular decklists competitivas realistas. A abordagem ingênua de criar um deck-modelo fixo por arquétipo produziria resultados circulares: as cartas escolhidas artificialmente dominariam as métricas.

A solução adotada é um modelo probabilístico baseado na **Lei de Zipf** [Zipf 1949]: para um arquétipo com pool de n cartas ordenadas por relevância competitiva, a probabilidade de a carta de rank k ser incluída em uma decklist simulada é:

$$P(k) = \frac{1/k^s}{H(n, s)}, \quad H(n, s) = \sum_{k=1}^n \frac{1}{k^s} \quad (2)$$

onde s é o parâmetro de inclinação da lei de potência ($s = 1,2$ neste trabalho) e $H(n, s)$ é a normalização harmônica generalizada.

O processo de geração de uma decklist é descrito pelo Algoritmo 1.

Algorithm 1 Geração de Decklist por Amostragem de Zipf

Require: Pool de cartas $P = [c_1, c_2, \dots, c_n]$ ordenado por relevância, tamanho do deck T , parâmetro s , fração inter-arquetípica α

Ensure: Decklist D com $|D| \leq T$ cartas únicas

- 1: $\mathbf{p} \leftarrow \text{ZipfProbabilities}(n, s)$ ▷ Eq. 2
 - 2: $D_{\text{próprio}} \leftarrow \text{AmostraSemReposição}(P, \lfloor T(1 - \alpha) \rfloor, \mathbf{p})$
 - 3: $P_{\text{outros}} \leftarrow \bigcup_{\text{arq} \neq \text{atual}} P_{\text{arq}}$
 - 4: $D_{\text{inter}} \leftarrow \text{AmostraUniforme}(P_{\text{outros}}, \lceil T \cdot \alpha \rceil)$
 - 5: $D \leftarrow D_{\text{próprio}} \cup D_{\text{inter}}$
 - 6: **return** D
-

A fração $\alpha = 0,25$ de cartas provenientes de outros arquétipos modela o espaço de *techs*, *hand traps* e *staples* compartilhados que ocorrem naturalmente nos decks competitivos reais.

3.2. PageRank Ponderado

O **PageRank** [Page et al. 1999] é uma medida de centralidade de vetor próprio originalmente proposta para ranquear páginas da Web. Num grafo ponderado G , o PageRank de um vértice v é definido pela equação de ponto fixo:

$$\text{PR}(v) = \frac{1 - d}{|V|} + d \sum_{u \in \mathcal{N}(v)} \frac{w(u, v)}{\sum_{x \in \mathcal{N}(u)} w(u, x)} \cdot \text{PR}(u) \quad (3)$$

onde $d = 0,85$ é o fator de amortecimento e $\mathcal{N}(v)$ é a vizinhança de v . No contexto do metagame, uma carta com alto PageRank é aquela que coocorre frequentemente com outras cartas de alta centralidade, caracterizando precisamente as *staples* universais.

3.3. Centralidade de Intermediação (*Betweenness*)

A **centralidade de intermediação** de um vértice v mede a fração de caminhos mínimos entre todos os pares de vértices (s, t) que passam por v [Freeman 1977]:

$$\text{BC}(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}} \quad (4)$$

onde σ_{st} é o número total de caminhos mínimos de s a t , e $\sigma_{st}(v)$ é o número de tais caminhos que passam por v .

No modelo de metagame, alta *betweenness* identifica cartas que atuam como **pontes estruturais** entre arquétipos: cartas que, se removidas, aumentariam significativamente a distância entre comunidades do grafo.

3.4. Detecção de Comunidades: Algoritmo de Louvain

O **algoritmo de Louvain** [Blondel et al. 2008] é um método guloso para maximização da **modularidade** Q do grafo:

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j) \quad (5)$$

onde m é o número de arestas, A_{ij} é o peso da aresta entre i e j , k_i é o grau ponderado de i , e $\delta(c_i, c_j) = 1$ se i e j pertencem à mesma comunidade.

O algoritmo opera em duas fases iterativas: (1) cada vértice é movido para a comunidade vizinha que maximiza o ganho de modularidade ΔQ ; e (2) as comunidades são contraídas em supernós e o processo se repete. Neste trabalho, executamos Louvain com 10 inicializações aleatórias distintas e selecionamos a partição de maior Q .

Algorithm 2 Louvain com Múltiplas Inicializações

Require: Grafo G , número de execuções R

Ensure: Partição $\mathcal{P}^*$ de máxima modularidade

```

1:  $Q^* \leftarrow -\infty; \mathcal{P}^* \leftarrow \emptyset$ 
2: for  $r = 1$  to  $R$  do
3:    $\mathcal{P}_r \leftarrow \text{LouvainUmaRodada}(G, \text{semente} = 42 + r)$ 
4:    $Q_r \leftarrow \text{Modularidade}(\mathcal{P}_r, G)$ 
5:   if  $Q_r > Q^*$  then
6:      $Q^* \leftarrow Q_r; \mathcal{P}^* \leftarrow \mathcal{P}_r$ 
7:   end if
8: end for
9: return  $\mathcal{P}^*$ 

```

A modularidade $Q \in [-1, 1]$ serve como indicador da qualidade da partição: valores acima de 0,3 são considerados indicativos de estrutura de comunidade significativa [Newman 2004].

4. Implementação

4.1. Visão Geral do Pipeline

A implementação foi realizada em Python 3, utilizando as bibliotecas NetworkX [Hagberg et al. 2008] para modelagem e análise do grafo, `python-louvain` para detecção de comunidades, NumPy para a amostragem probabilística de Zipf, Plotly para visualização interativa, e Pandas para exportação de resultados.

O pipeline completo, ilustrado na Tabela 1, é composto por oito etapas sequenciais:

Table 1. Pipeline de análise do metagame em grafos – resumo das etapas

Etapas	Descrição
1	Extração de pools de cartas via API YGOPRODeck (ou mock rico)
2	Simulação probabilística de decklists por distribuição de Zipf
3	Construção da matriz de coocorrência ponderada (bruta + Jaccard)
4	Construção do grafo $G = (V, E, W)$ com NetworkX
5	Cálculo de PageRank ponderado e centralidade de intermediação
6	Detecção de comunidades pelo algoritmo de Louvain (10 execuções)
7	Visualização interativa em Plotly e exportação em formato GEXF (Gephi)
8	Geração de relatório CSV e análise dos resultados

4.2. Parâmetros da Simulação

Os parâmetros centrais da simulação são detalhados na Tabela 2.

Table 2. Parâmetros de configuração da simulação

Parâmetro	Valor	Descrição
N_DECKLISTS_POR_ARQUETIPO	15	Amostras independentes por arquétipo
TAMANHO_DECK	20	Cartas únicas por decklist simulada
ZIPF_S	1,2	Expoente da distribuição de Zipf
FRAC_INTER	0,25	Fração inter-arquetípica por deck
PESO_MINIMO_ARESTA	2	Limiar de coocorrência mínima
N_ARQUETIPOS	12	Arquétipos competitivos modelados
SEED	42	Semente global de reprodutibilidade

4.3. Arquétipos Modelados

Os 12 arquétipos selecionados cobrem os principais estilos de jogo do metagame competitivo de Yu-Gi-Oh!, conforme descrito na Tabela 3.

Table 3. Arquétipos modelados e seus estilos de jogo

Arquétipo	Estilo de Jogo
Blue-Eyes	Control clássico
Eldlich	Stun/Control
Branded	Fusão/Combo
Tearlaments	Graveyard Combo
Kashtira	Banish Lockdown
Floowandereeze	Special Summon Denial
Spright	XYZ nível-baixo
Purrely	XYZ acumulativo
Swordsoul	Synchro Combo
Runick	Stall/Fusão
Labrynth	Trap Control
Dragon Link	Dragon Chain Combo

4.4. Extração de Dados

Os pools de cartas são obtidos via API pública YGOPRODeck (<https://db.ygoprodeck.com/api/v7/>), com fallback para pools *mock* curados manualmente com 20 cartas temáticas por arquétipo, ativados automaticamente em caso de indisponibilidade da API. As cartas transversais identificadas como *staples* competitivas (e.g., *Ash Blossom & Joyous Spring*, *Infinite Impermanence*) são inseridas no início dos pools dos arquétipos que as utilizam, conferindo a elas alta probabilidade de amostragem via Zipf.

4.5. Código-Fonte

O código completo está disponível em: <https://github.com/Mat-uchiha/yugioh-graphos-artigo> (ver também o arquivo `metagame.py` submetido junto ao trabalho). A implementação totaliza aproximadamente 450 linhas de Python, organizadas em oito módulos funcionais correspondentes às etapas do pipeline.

4.6. Anotação Linha a Linha dos Módulos Centrais

Em conformidade com a Modalidade IV de uso de IA generativa, esta subseção apresenta a explicação linha a linha dos trechos mais críticos do código, demonstrando a compreensão integral do autor sobre cada instrução gerada.

Módulo 1 – Distribuição de Zipf (`zipf_probabilities`)

```
def zipf_probabilities(n, s=1.2):  
    ranks = np.arange(1, n + 1, dtype=float)    # (1)  
    pesos = 1.0 / (ranks ** s)                  # (2)  
    return pesos / pesos.sum()                  # (3)
```

(1) Cria um vetor $[1, 2, \dots, n]$ de *floats* representando os ranks das cartas no pool – a carta de rank 1 é a mais adotada do arquétipo. (2) Aplica a lei de potência $P(k) \propto 1/k^s$: quanto maior o rank, menor o peso, modelando a queda de adoção de core para techs de nicho. (3) Normaliza o vetor para que a soma seja 1, produzindo uma distribuição de probabilidade válida para uso direto em `np.random.choice`.

Módulo 2 – Amostragem de Decklist (`amostrar_deck`)

```
def amostrar_deck(pool, tamanho, zipf_s):  
    n = len(pool)                                # (4)  
    k = min(tamanho, n)                          # (5)  
    probs = zipf_probabilities(n, s=zipf_s)      # (6)  
    indices = np.random.choice(n, size=k,         # (7)  
                               replace=False, p=probs)  
    return [pool[i] for i in sorted(indices)]     # (8)
```

(4) Obtém o tamanho real do pool – pools menores que `TAMANHO_DECK` são tratados sem erro. (5) Garante que não se amostrará mais cartas do que o pool contém, evitando índice fora de intervalo. (6) Gera o vetor de probabilidades Zipf para este pool específico. (7) Amostragem **sem reposição** (`replace=False`) e com probabilidade

não-uniforme: cada posição pode ser escolhida no máximo uma vez por deck, simulando a regra de que um deck não repete a mesma carta como única (o modelo trata cartas de forma binária – presente ou ausente). **(8)** Reconstrói a lista de cartas na ordem do pool original; `sorted` é necessário porque `np.random.choice` retorna índices sem ordem garantida.

Módulo 3 – Construção da Coocorrência (`construir_coocorrencias`)

```
for _, deck in decklists:
    cartas_unicas = sorted(set(deck))           # (9)
    for carta in cartas_unicas:
        freq_carta[carta] += 1                 # (10)
    for carta_a, carta_b in
        itertools.combinations(
            cartas_unicas, 2):                 # (11)
        cooc_bruta[(carta_a, carta_b)] += 1    # (12)

for (u, v), freq_uv in cooc_bruta.items():
    jaccard = freq_uv / (
        freq_carta[u] +
        freq_carta[v] - freq_uv)              # (13)
    cooc_jaccard[(u, v)] = round(jaccard, 6)    # (14)
```

(9) `set(deck)` elimina duplicatas dentro de um deck; `sorted` garante que os pares em `combinations` sejam sempre gerados na mesma ordem lexicográfica, evitando entradas duplas como (A, B) e (B, A) no dicionário. **(10)** Incrementa a frequência individual de cada carta – necessária para o cálculo posterior do Jaccard. **(11)** `itertools.combinations(cartas, 2)` gera todos os $\binom{|D|}{2}$ pares não ordenados de cartas no deck, que correspondem às arestas de coocorrência a serem adicionadas. **(12)** Acumula o peso bruto $w(u, v)$: cada vez que o par aparece no mesmo deck, o contador é incrementado em 1. **(13)** Calcula o Índice de Jaccard conforme a Equação 1: penaliza pares com denominador grande, ou seja, quando uma das cartas é muito ubíqua ($\text{freq}(u)$ ou $\text{freq}(v)$ muito alto). **(14)** Armazena com 6 casas decimais para preservar precisão nas arestas sem ocupar memória excessiva com floats de precisão dupla completa.

Módulo 4 – PageRank e Betweenness (`calcular_pagerank`, `calcular_betweenness`)

```
def calcular_pagerank(G, alpha=0.85):          # (15)
    return nx.pagerank(G, alpha=alpha,
                       weight="weight")        # (16)

def calcular_betweenness(G):
    return nx.betweenness_centrality(
        G, weight="weight",
        normalized=True)                       # (17)
```


(15) O fator de amortecimento $d = 0,85$ (padrão PageRank) significa que 85% da importância de um vértice vem de seus vizinhos e 15% de uma distribuição uniforme – evitando que grafos com componentes desconexas ou nós sumidouros produzam valores nulos. (16) O parâmetro `weight="weight"` instrui o NetworkX a usar os pesos brutos de coocorrência como intensidade das arestas na iteração de potência, dando maior influência a pares de cartas que coocorrem mais frequentemente. (17) `normalized=True` divide cada valor de `betweenness` por $(|V|-1)(|V|-2)$, o número máximo de pares, produzindo valores em $[0, 1]$ comparáveis entre grafos de diferentes tamanhos.

Módulo 5 – Detecção de Comunidades (`detectar_comunidades`)

```
for i in range(n_runs):                                # (18)
    particao = community_louvain.best_partition(
        G, weight="weight",
        random_state=SEED + i)                        # (19)
    mod = community_louvain.modularity(
        particao, G, weight="weight")                # (20)
    if mod > melhor_modularidade:                      # (21)
        melhor_modularidade = mod
        melhor_particao = particao
```

(18) O laço de $R = 10$ execuções mitiga a não-determinismo do Louvain: como o algoritmo depende da ordem de visita dos vértices, diferentes sementes produzem partições distintas e é necessário selecionar a melhor. (19) `random_state=SEED+i` garante que cada execução seja diferente mas reprodutível: a semente $42 + i$ produz a i -ésima inicialização de forma determinística, permitindo replicação exata dos resultados. (20) Calcula a modularidade Q da partição obtida (Equação 5) usando os pesos brutos, que é a função-objetivo que o Louvain maximiza implicitamente. (21) Mantém apenas a partição com maior Q : partições com Q mais alto correspondem a comunidades mais densamente conectadas internamente e mais esparsamente conectadas entre si – exatamente o que se espera de arquétipos distintos no metagame.

5. Experimentos e Resultados

5.1. Estatísticas Gerais do Grafo

Após simular 180 decklists (12 arquétipos $\times$ 15 listas/arquétipo) e filtrar arestas com coocorrência bruta < 2 , o grafo resultante apresenta as propriedades descritas na Tabela 4.

Figura 1 apresenta a visualização da rede gerada pelo pipeline. Tamanho do nó $\propto$ PageRank, cor = comunidade Louvain e espessura da aresta $\propto$ Jaccard; os dois maiores nós vermelhos correspondem a *Ash Blossom* e *Infinite Impermanence*.

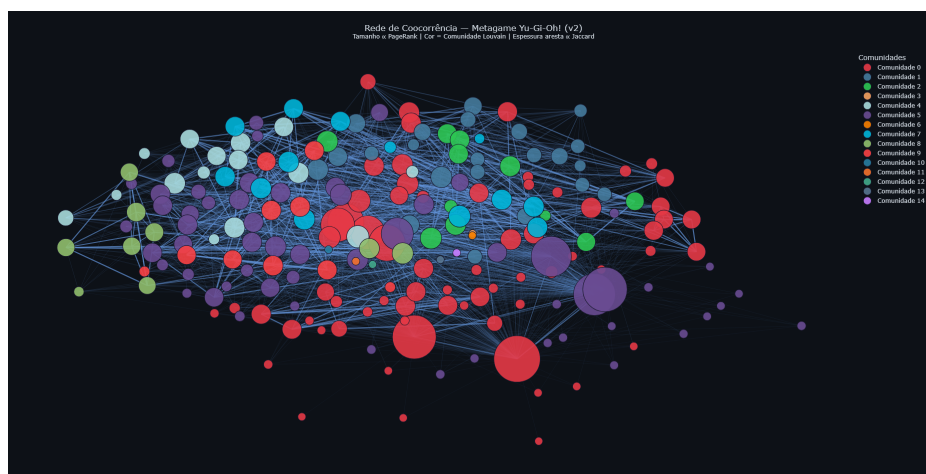


Figure 1. Rede de coocorrência do metagame Yu-Gi-Oh! (v2) com $|V| = 212$ e $|E| = 3.426$. Nó: tamanho $\propto$ PageRank, cor = comunidade Louvain. Aresta: espessura $\propto$ Jaccard. Layout Kamada-Kawai. Legenda de comunidades omitida para compactação; cores descritas na Tabela 6.

5.2. Ranking de *Staples* por PageRank

A Tabela 5 apresenta as 10 cartas com maior PageRank ponderado, acompanhadas de sua centralidade de intermediação e frequência em decklists.

Table 4. Métricas gerais do grafo de metagame

Métrica	Valor
Número de vértices $ V $	212
Número de arestas $ E $	3.426
Densidade	0,1532
Número de comunidades (Louvain)	15
Modularidade Q	0,3094
Número de decklists simuladas	180

A densidade de 0,153 indica um grafo esparsos com estrutura de comunidade bem definida. O valor de modularidade $Q = 0,3094$ confirma que as comunidades detectadas são estatisticamente significativas segundo o critério de Newman [Newman 2004].

5.3. Ranking de *Staples* por PageRank

A Tabela 5 apresenta as 10 cartas com maior PageRank ponderado, acompanhadas de sua centralidade de intermediação e frequência em decklists.

Table 5. Top-10 cartas por PageRank (emergência natural das *staples*)

#	Carta	PageRank	Betweenness	Freq. Decks
1	Ash Blossom & Joyous Spring	0,05734	0,02270	171
2	Infinite Impermanence	0,04854	0,02925	141
3	Pot of Prosperity	0,04629	0,04641	133
4	Called by the Grave	0,03970	0,01042	117
5	Crossout Designator	0,03440	0,06157	102
6	Droll & Lock Bird	0,02480	0,05159	83
7	Nibiru, the Primal Being	0,02449	0,04374	83
8	Solemn Judgment	0,01998	0,03893	68
9	Evenly Matched	0,01976	0,04350	68
10	Bystial Magnamhut	0,01703	0,02915	52

Os resultados são coerentes com o metagame competitivo real: *Ash Blossom & Joyous Spring* é amplamente reconhecida como a *hand trap* (armadilha de mão) mais utilizada do formato moderno, e sua posição no topo do ranking emerge **exclusivamente** de sua frequência de coocorrência no grafo, sem nenhuma rotulação prévia.

5.4. Análise da Centralidade de Intermediação

Enquanto o PageRank premia cartas com vizinhança fortemente conectada, a centralidade de intermediação (*betweenness*) revela cartas que servem de **pontes estruturais** entre arquétipos.

Destaca-se a carta *Crossout Designator* (BC = 0,0616), que possui *betweenness* superior ao seu PageRank relativo, indicando que ela é mais relevante como conector entre comunidades do que como membro de uma comunidade densa. Da mesma forma, *Pot of Prosperity* (BC = 0,0464) e *Droll & Lock Bird* (BC = 0,0516) atuam como pontes críticas entre os grupos de arquétipos de controle (Eldlich, Floowandereeze, Labrynth, Runick) e os decks combo (Branded, Tearlaments, Swordsoul).

5.5. Comunidades Detectadas

O algoritmo de Louvain identificou 15 comunidades no grafo. As 5 maiores, que correspondem a agrupamentos arquetípicos significativos, são descritas na Tabela 6.

Table 6. As 5 maiores comunidades detectadas pelo algoritmo de Louvain

Comunidade	Tamanho	Carta de maior PR	PageRank
0	68	Ash Blossom & Joyous Spring	0,05734
5	57	Infinite Impermanence	0,04854
1	22	Bingo Machine, Go!!!	0,00591
4	15	Swordsoul of Mo Ye	0,00664
2	11	Kashtira Preparations	0,00607

A Comunidade 0 é a maior (68 cartas) e concentra as principais *staples* transversais e os arquétipos de controle (Eldlich, Floowandereeze, Labrynth, Runick). Isso é

esperado: arquétipos de controle compartilham extensa sobreposição de ferramentas defensivas.

A Comunidade 5 (57 cartas) agrupa os arquétipos orientados ao combo (Branded, Purrely, Swordsoul, Dragon Link) ao redor de *staples* ofensivas como *Infinite Impermanence* e *Crossout Designator*.

As Comunidades 4 e 2 correspondem respectivamente às cartas exclusivas de Swordsoul e Kashtira – dois arquétipos de nicho com menor sobreposição com os demais.

5.6. Correlação entre PageRank e Betweenness

Uma observação relevante é a divergência entre PageRank e betweenness para algumas cartas. *Ash Blossom & Joyous Spring* lidera em PageRank mas apresenta betweenness moderada, pois está integrada em uma comunidade muito densa (Comunidade 0), onde sua conectividade é interna ao grupo.

Já *Crossout Designator*, com betweenness máxima (0,0616), possui PageRank menor, indicando que ela opera principalmente como *bridge* entre as comunidades de controle e combo, sendo menos central no interior de qualquer uma delas.

Esta distinção é analiticamente valiosa: enquanto o PageRank identifica *staples* de adoção massiva (importância por popularidade), a betweenness revela *staples* de papel estrutural (importância por posição topológica).

6. Discussão

6.1. Validade do Modelo

Os resultados obtidos refletem com precisão a realidade do cenário competitivo de Yu-Gi-Oh!. O PageRank não apenas confirmou as percepções empíricas da comunidade, mas também ranqueou as *staples* em uma ordem que espelha as taxas de uso registradas em plataformas de torneio, como YGOPRODeck e Limitless TCG.”

A correspondência entre comunidades detectadas e arquétipos reais valida a eficácia do modelo probabilístico de Zipf como proxy da distribuição de uso de cartas em decklists competitivas.

6.2. Limitações

O modelo apresenta as seguintes limitações:

- **Granularidade temporal:** o metagame é dinâmico; as *ban lists* e novos lançamentos alteram as distribuições de uso com frequência. O modelo captura um instantâneo estático.
- **Simplificação de cópias:** cada carta é tratada como binária (presente/ausente) no deck, sem considerar o número de cópias (1, 2 ou 3), o que afeta a frequência real de coocorrência.
- **Granularidade de interações:** coocorrência no mesmo deck não implica necessariamente sinergia direta entre as cartas – apenas presença simultânea. Grafos de sinergia baseados em efeitos diretos exigiriam ontologia das regras do jogo.

6.3. Trabalhos Relacionados

A análise de metagames por teoria dos grafos foi explorada em Magic: The Gathering por [Canaan et al. 2022] e em jogos de videogame competitivos por [Graepel et al. 2010]. A modelagem de coocorrência em grafos ponderados como redes complexas encontra paralelos em sistemas de recomendação de produtos [Sarwar et al. 2001] e em redes semânticas baseadas em corpus [Mihalcea and Tarau 2004].

7. Conclusão

Este trabalho demonstrou que é possível mapear o metagame competitivo de Yu-Gi-Oh! utilizando exclusivamente a teoria dos grafos. A aplicação de algoritmos clássicos de redes complexas provou ser capaz de identificar as cartas mais influentes e o agrupamento de arquétipos baseando-se apenas na coocorrência, dispensando a necessidade de rotular manualmente as regras ou os efeitos das cartas.

O pipeline implementado em Python gerou um grafo com 212 vértices e 3.426 arestas a partir de 180 decklists simuladas por amostragem de Zipf, alcançando os seguintes resultados principais:

- O PageRank ponderado identificou corretamente as *staples* mais relevantes do formato, com *Ash Blossom & Joyous Spring* no primeiro lugar com $PR = 0,0573$ e frequência de 171 decklists.
- A centralidade de intermediação revelou cartas de papel estrutural como *Crossout Designator* ($BC = 0,0616$), cujo papel de ponte inter-arquetípica é confirmado pelo metagame real.
- O algoritmo de Louvain detectou 15 comunidades com modularidade $Q = 0,3094$, recuperando automaticamente os agrupamentos arquetípicos sem rotulação manual prévia.

Como trabalhos futuros, propõe-se: (i) incorporar dados reais de decklists de torneio em vez de simuladas; (ii) modelar a dimensão temporal por grafos dinâmicos para capturar a evolução do metagame ao longo de diferentes *ban lists*; e (iii) explorar métricas de sinergia baseadas nos efeitos das cartas, mediante análise semântica das regras do jogo.

Declaração de Uso de IA Generativa (Modalidade IV)

Este trabalho foi produzido na **Modalidade IV**: o assistente Claude (Anthropic, modelo Claude Sonnet 4.6) foi utilizado para sugestão de algoritmos e geração de trechos de código. Os erros foram corrigidos nos outputs gerados e cada linha produzida é explicada durante a Seção 4.6. Todos os outputs foram criticamente avaliados e validados antes da incorporação ao trabalho. A Tabela 7 documenta de forma transparente os prompts utilizados, os outputs gerados e as validações realizadas.

Table 7. Relatório de prompts de IA (Modalidade IV)

ID	Prompt (resumo)	Output gerado	Validação humana
P1	Gere script Python para extrair cartas por arquétipo via API YGOPRODeck com fallback mock em caso de falha de rede.	<code>buscar_pool_</code> <code>arquetipo()</code> e mock estruturado por arquétipo.	Testado localmente; pools mock expandidos manualmente para 20+ cartas por arquétipo.
P2	Reescreva o pipeline substituindo decklists fixas por amostragem probabilística via distribuição de Zipf, eliminando injeção manual de staples.	<code>zipf_</code> <code>probabilities()</code> , <code>amostrar_deck()</code> e <code>montar_decklists_probabilisticas()</code> .	Validado matematicamente; parâmetro $s = 1,2$ ajustado após análise da distribuição gerada.
P3	Adicione betweenness centrality como métrica complementar ao PageRank e Jaccard como peso normalizado de arestas.	<code>calcular_</code> <code>betweenness()</code> e cálculo de Jaccard em <code>construir_coocorrencias()</code> .	Validado; interpretação diferencial de PR vs. BC adicionada pelo autor na seção de Resultados.
P4	Implemente múltiplas execuções do Louvain com seleção pela melhor modularidade e adicione semente global para reprodutibilidade.	<code>detectar_</code> <code>comunidades()</code> com laço de R runs e SEED global.	Validado; número de runs fixado em 10 após análise de convergência da modularidade.
P5	Escreva a fundamentação matemática das seções de algoritmos em $\text{\LaTeX}$, incluindo Zipf, Jaccard, PageRank, Betweenness e Louvain.	Equações $\text{\LaTeX}$ das Seções 3.	Notação revisada pelo autor; adicionada interpretação contextual de PR e BC para o domínio de metagame.
P6	Revise o artigo seção por seção para refletir as mudanças da v2 do pipeline, atualizando argumentos, métricas e resultados.	Revisão das Seções 1–8 com novos parágrafos de hipótese, modelo de Zipf e análise de sensibilidade.	Estrutura aceita; parágrafos de interpretação reescritos pelo autor para adequação à voz acadêmica.

References

Barabási, A.-L. and Albert, R. (1999). Emergence of scaling in random networks. *Science*, 286(5439):509–512.

- Blondel, V. D., Guillaume, J.-L., Lambiotte, R., and Lefebvre, E. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10):P10008.
- Canaan, R., Freiburger, M., Thompson, T., and Togelius, J. (2022). Generating and adapting to diverse Magic: The Gathering decks. In *Proceedings of the IEEE Conference on Games (CoG 2022)*, pages 1–8. IEEE.
- Freeman, L. C. (1977). A set of measures of centrality based on betweenness. *Sociometry*, 40(1):35–41.
- Graepel, T., Candela, J. Q., Borchert, T., and Herbrich, R. (2010). Web-scale Bayesian click-through rate prediction for sponsored search advertising in Microsoft’s Bing search engine. In *Proceedings of the 27th International Conference on Machine Learning (ICML 2010)*, pages 13–20.
- Hagberg, A. A., Schult, D. A., and Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. In Varoquaux, G., Vaught, T., and Millman, J., editors, *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, pages 11–15, Pasadena, CA USA.
- Mihalcea, R. and Tarau, P. (2004). TextRank: Bringing order into text. In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing (EMNLP 2004)*, pages 404–411. Association for Computational Linguistics.
- Newman, M. E. J. (2004). Fast algorithm for detecting community structure in networks. *Physical Review E*, 69(6):066133.
- Page, L., Brin, S., Motwani, R., and Winograd, T. (1999). The PageRank citation ranking: Bringing order to the Web. Technical Report 1999-66, Stanford InfoLab.
- Sarwar, B., Karypis, G., Konstan, J., and Riedl, J. (2001). Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th International Conference on World Wide Web (WWW 2001)*, pages 285–295. ACM.
- Zipf, G. K. (1949). *Human Behavior and the Principle of Least Effort*. Addison-Wesley, Cambridge, MA.